

## Zadanie

### SIMPLECHAT\_NIO

Napisać serwer czatu, który:

- obsługuje logowanie klientów (tylko id, bez hasła),
- przyjmuje od klientów wiadomości i rozsyła je do zalogowanych klientów,
- obsługuje wylogowanie klientów,
- gromadzi wszystkie odpowiedzi na żądania klientów w logu, realizowanym w pamięci wewnętrznej (poza systemem plikowym).

Zadania te wykonuje klasa **ChatServer**, która ma:

- konstruktor: **public ChatServer(String host, int port)**
- metodę: **public void startServer()**, która uruchamia serwer w odrębnym wątku,
- metodę: **public void stopServer()**, która zatrzymuje serwer i wątek, w którym działa,
- metodę **String getServerLog()** - zwraca log serwera (wymagany format logu będzie widoczny w dalszych przykładach).

Wymagania konstrukcyjne dla klasy ChatServer:

- multipleksowania kanałów gniazd (użycie selektora),
- serwer może obsługiwać równolegle wielu klientów, ale obsługa żądań klientów odbywa się w jednym wątku,

Dostarczyć także klasy **ChatClient** z konstruktorem:

**public ChatClient(String host, int port, String id)**, gdzie id - id klienta

i następującymi metodami:

- **public void login()** - loguje klienta na serwer
- **public void logout()** - wylogowuje klienta,
- **public void send(String req)** - wysyła do serwera żądanie req
- **public String getChatView()** - zwraca dotychczasowy widok czatu z pozycji danego klienta (czyli wszystkie informacje, jakie dostaje on po kolei od serwera)

Dla metody *send* żądaniem może być posłanie tekstu wiadomości, zalogowanie, wylogowanie, a protokół komunikacji z serwerem można po swojemu wymyślić.

Wymagania konstrukcyjne dla klasy ChatClient

- nieblokujące wejście - wyjście

Dodatkowo stworzyć klasę **ChatClientTask**, umożliwiającą uruchamianie klientów w odrębnych wątkach poprzez ExecutorService.

Obiekty tej klasy tworzy statyczna metoda:

**public static ChatClientTask create(Client c, List<String> msgs, int wait)**

gdzie:

c - klient (obiekt klasy Client)

msgs - lista wiadomości do wysłania przez klienta c

wait - czas wstrzymania pomiędzy wysyłaniem żądań.

Kod działający w wątku ma wykonywać następując działania:

- łączy się z serwerem i loguje się (c.login())
- wysyła kolejne wiadomości z listy msgs (c.send(...))
- wylogowuje klienta (c.logout())

Parametr *wait* w sygnaturze metody *create* oznacza czas w milisekundach, na jaki wątek danego klienta jest wstrzymywany po każdym żądaniu. Jeśli *wait* jest 0, wątek klienta nie jest wstrzymywany,

Oto pseudokod fragmentu odpowiedzialnego za posyłanie żądań:

```

c.login();
if (wait != 0) uśpienie_watku_na wait ms;
// ....
dla_każdej_wiadomości_z_listy msgs {
    // ...
    c.send( wiadomość );
    if (wait != 0) uśpienie_watku_na wait ms;
}
c.logout();
if (wait != 0) uśpienie_watku_na wait ms;

```

W projekcie znajduje się klasa **Main (plik niemodyfikowalny)**, w której z pliku testowego wprowadzane są informacje nt. konfiguracji serwera (host, port) oraz klientów (id, czas wstrzymania wątku po każdym żądaniu, zestaw wiadomości do wystania).

Format pliku testowego:

pierwszy wiersz: nazwa\_hosta

drugi wiersz: nr portu

kolejne wiersze:

id\_klienta<TAB>parametr wait w ms<TAB>msg1<TAB>msg2<TAB> .... <TAB>msgN

Klasa Main:

```

import java.io.*;
import java.nio.file.*;
import java.util.*;
import java.util.concurrent.*;

public class Main {

    public static void main(String[] args) throws Exception {

        String testFileName = System.getProperty("user.home") + "/ChatTest.txt";
        List<String> test = Files.readAllLines(Paths.get(testFileName));
        String host = test.remove(0);
        int port = Integer.valueOf(test.remove(0));
        ChatServer s = new ChatServer(host, port);
        s.startServer();

        ExecutorService es = Executors.newCachedThreadPool();
    }
}

```

```

List<ChatClientTask> ctasks = new ArrayList<>();

for (String line : test) {
    String[] elts = line.split("\t");
    String id = elts[0];
    int wait = Integer.valueOf(elts[1]);
    List<String> msgs = new ArrayList<>();
    for (int i = 2; i < elts.length; i++) msgs.add(elts[i] + ", mówię ja, " + id);
    ChatClient c = new ChatClient(host, port, id);
    ChatClientTask ctask = ChatClientTask.create(c, msgs, wait);
    ctasks.add(ctask);
    es.execute(ctask);
}
ctasks.forEach( task -> {
    try {
        task.get();
    } catch (InterruptedException | ExecutionException exc) {
        System.out.println("*** " + exc);
    }
});
es.shutdown();
s.stopServer();

System.out.println("\n=== Server log ===");
System.out.println(s.getServerLog());

ctasks.forEach(t -> System.out.println(t.getClient().getChatView()));
}
}

```

Dla pliku testowego o treści:

localhost

9999

Asia 50 Dzień dobry aaaa bbbb Do widzenia

Adam 50 Dzień dobry aaaa bbbb Do widzenia

Sara 50 Dzień dobry aaaa bbbb Do widzenia

program ten może wyprowadzić:

Server started

Server stopped

=== Server log ===

00:25:08.698 Asia logged in  
00:25:08.698 Sara logged in  
00:25:08.745 Adam logged in  
00:25:08.745 Sara: Dzień dobry, mówię ja, Sara  
00:25:08.745 Asia: Dzień dobry, mówię ja, Asia  
00:25:08.807 Sara: aaaa, mówię ja, Sara  
00:25:08.807 Adam: Dzień dobry, mówię ja, Adam  
00:25:08.807 Asia: aaaa, mówię ja, Asia  
00:25:08.869 Adam: aaaa, mówię ja, Adam  
00:25:08.869 Sara: bbbb, mówię ja, Sara  
00:25:08.869 Asia: bbbb, mówię ja, Asia  
00:25:08.932 Adam: bbbb, mówię ja, Adam  
00:25:08.932 Sara: Do widzenia, mówię ja, Sara  
00:25:08.932 Asia: Do widzenia, mówię ja, Asia  
00:25:08.994 Sara logged out  
00:25:08.994 Adam: Do widzenia, mówię ja, Adam  
00:25:08.994 Asia logged out  
00:25:09.057 Adam logged out

=== Asia chat view

Asia logged in  
Sara logged in  
Adam logged in  
Sara: Dzień dobry, mówię ja, Sara  
Asia: Dzień dobry, mówię ja, Asia  
Sara: aaaa, mówię ja, Sara  
Adam: Dzień dobry, mówię ja, Adam  
Asia: aaaa, mówię ja, Asia  
Adam: aaaa, mówię ja, Adam  
Sara: bbbb, mówię ja, Sara  
Asia: bbbb, mówię ja, Asia  
Adam: bbbb, mówię ja, Adam  
Sara: Do widzenia, mówię ja, Sara  
Asia: Do widzenia, mówię ja, Asia  
Sara logged out

Adam: Do widzenia, mówię ja, Adam  
Asia logged out

=== Adam chat view

Adam logged in  
Sara: Dzień dobry, mówię ja, Sara  
Asia: Dzień dobry, mówię ja, Asia  
Sara: aaaa, mówię ja, Sara  
Adam: Dzień dobry, mówię ja, Adam  
Asia: aaaa, mówię ja, Asia  
Adam: aaaa, mówię ja, Adam  
Sara: bbbb, mówię ja, Sara  
Asia: bbbb, mówię ja, Asia  
Adam: bbbb, mówię ja, Adam  
Sara: Do widzenia, mówię ja, Sara  
Asia: Do widzenia, mówię ja, Asia  
Sara logged out  
Adam: Do widzenia, mówię ja, Adam  
Asia logged out  
Adam logged out

Przykład pokazuje wymaganą formę wydruku (chatView klienta, wejścia w logu serwera).  
W szczególności:

- start serwera powoduje wypisanie na konsoli: Server started
- logowanie klienta *id* skutkuje rozestaniem wiadomości: ***id* logged in**
- wylogowanie klienta *id* skutkuje rozestaniem wiadomości: ***id* logged out**
- otrzymanie wiadomości *msg* od klienta *id* skutkuje rozesłaniem wiadomości ***id: msg***
- widok czatu zwracany przez client.getChatView() dla klienta *id* jest poprzedzony nagłówkiem: **=== *id* chat view**
- log serwera jest poprzedzony nagłówkiem **=== Server log ===** i zawiera kolejno wszystkie odpowiedzi serwera z podaniem czasu w formacie HH:MM:SS.nnn, gdzie nnn - milisekundy (czas wg zegara systemowego),
- zatrzymanie serwera wypisuje na konsoli: Server stopped

- wszelkie błędy w interakcji klienta z serwerem (wyjątki exc, np. IOException) powinny być dodawane do chatView klienta jako exc.toString() poprzedzone trzema gwiazdkami

Forma wydruku jest obowiązkowa, a jej niedotrzymanie powoduje utratę punktów. Konkretna zawartość wydruków chatview i logu seerwera (kolejność wierszy, podane czasy itp.) mogą być w każdym przebiegu inne, ważne jednak, aby widac było równoległą obsługę klientów oraz zachowaną logikę: klienci otrzymują, w odpowiedniej kolejności, tylko te wiadomości, które się pojawiły od momentu ich logowania do momentu wylogowania.

### **Podsumowanie:**

trzeba stworzyć klasy ChatServer, ChatClient, ChatClientTask w taki sposób, aby zapewnić właściwe wykonanie kodu metody main z klasy Main

### **Ale**

trzeba przygotować kod na rozliczne konfiguracje podawane w pliku ChatTest.txt. Przykłady wyników Main.main():

Dla:

localhost

33333

Asia 20 Dzień dobry beee Do widzenia

Sara 20 Dzień dobry muuu Do widzenia

Wynik:

Server started

Server stopped

=== Server log ===

01:18:43.723 Asia logged in

01:18:43.723 Sara logged in

01:18:43.738 Asia: Dzień dobry, mówię ja, Asia

01:18:43.738 Sara: Dzień dobry, mówię ja, Sara

01:18:43.769 Sara: muuu, mówię ja, Sara

01:18:43.769 Asia: beee, mówię ja, Asia

01:18:43.801 Asia: Do widzenia, mówię ja, Asia

01:18:43.801 Sara: Do widzenia, mówię ja, Sara

01:18:43.832 Asia logged out

01:18:43.832 Sara logged out

=== Asia chat view

Asia logged in

Sara logged in

Asia: Dzień dobry, mówię ja, Asia

Sara: Dzień dobry, mówię ja, Sara

Sara: muuu, mówię ja, Sara

Asia: beee, mówię ja, Asia

Asia: Do widzenia, mówię ja, Asia

Sara: Do widzenia, mówię ja, Sara

Asia logged out

=== Sara chat view

Sara logged in

Asia: Dzień dobry, mówię ja, Asia

Sara: Dzień dobry, mówię ja, Sara

Sara: muuu, mówię ja, Sara

Asia: beee, mówię ja, Asia

Asia: Do widzenia, mówię ja, Asia

Sara: Do widzenia, mówię ja, Sara

Asia logged out

Sara logged out

Dla:

localhost

55557

Asia 10 Dzień dobry beee Do widzenia

Sara 20 Dzień dobry muuu Do widzenia

Server started

Server stopped

=== Server log ===

01:25:33.293 Asia logged in

01:25:33.293 Asia: Dzień dobry, mówię ja, Asia

01:25:33.308 Asia: beee, mówię ja, Asia

01:25:33.324 Sara logged in

01:25:33.324 Asia: Do widzenia, mówię ja, Asia

01:25:33.339 Asia logged out

01:25:33.355 Sara: Dzień dobry, mówię ja, Sara



01:25:33.386 Sara: muuu, mówię ja, Sara  
01:25:33.417 Sara: Do widzenia, mówię ja, Sara  
01:25:33.449 Sara logged out

=== Asia chat view

Asia logged in  
Asia: Dzień dobry, mówię ja, Asia  
Asia: beee, mówię ja, Asia  
Sara logged in  
Asia: Do widzenia, mówię ja, Asia  
Asia logged out

=== Sara chat view

Sara logged in  
Asia: Do widzenia, mówię ja, Asia  
Asia logged out  
Sara: Dzień dobry, mówię ja, Sara  
Sara: muuu, mówię ja, Sara  
Sara: Do widzenia, mówię ja, Sara  
Sara logged out

A dla takiej konfiguracji (zagłódzenie wątku):

localhost

55557

Asia 0 Dzień dobry beee Do widzenia  
Sara 0 Dzień dobry muuu Do widzenia

Wynik:

Server started

Server stopped

=== Server log ===

01:50:13.603 Asia logged in  
01:50:13.603 Asia: Dzień dobry, mówię ja, Asia  
01:50:13.603 Asia: beee, mówię ja, Asia  
01:50:13.603 Asia: Do widzenia, mówię ja, Asia  
01:50:13.603 Asia logged out  
01:50:13.634 Sara logged in  
01:50:13.634 Sara: Dzień dobry, mówię ja, Sara  
01:50:13.634 Sara: muuu, mówię ja, Sara

01:50:13.634 Sara: Do widzenia, mówię ja, Sara

01:50:13.634 Sara logged out

=== Asia chat view

Asia logged in

Asia: Dzień dobry, mówię ja, Asia

Asia: beee, mówię ja, Asia

Asia: Do widzenia, mówię ja, Asia

Asia logged out

=== Sara chat view

Sara logged in

Sara: Dzień dobry, mówię ja, Sara

Sara: muuu, mówię ja, Sara

Sara: Do widzenia, mówię ja, Sara

Sara logged out

**Uwaga: plik Main.java jest niemodyfikowalny.**